Michelle O. Gadayan

Jasmin Kim S. Quisel

SECTION C

NOVEMBER 11, 2026

Campus Café Ordering System

Concept Explanation

If Statement

We used if statements in the Pastry and Meal classes to check if a pastry is warmed or if a meal is upsized. When the condition is true, an additional amount is added to the original price.

If-else Statement

We used if-else statements to check and validate information in our program. For example, we used them when setting the base price, calculating beverage prices, checking the order limit, and applying student discounts.

Classes and Objects

We created seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. Each class has its own purpose, and we used objects to make the program work.

Attributes and Behaviors

We used attributes and methods in every class. For example, the Customer class has a name, student status, and loyalty points. It also has methods that allow us to add points and update customer information.

Object Interaction

We applied object interaction in the Order class because it connects the Customer and MenuItem objects. This allows our program to get customer information, add menu items, calculate prices, and update loyalty points.

Java Naming Conventions

We followed Java naming conventions by using capital letters at the beginning of class names, such as Customer and Beverage. For methods and variables, we used camelCase, such as computePrice() and itemCount.

Default Constructor

We used default constructors to give initial values to objects when no information is provided. These can be found in the MenuItem, Beverage, Pastry, Meal, and Customer classes.

Parameterized Constructor

We used parameterized constructors to assign specific information when creating objects, such as the customer's name, menu item name, price, and item code.

Constructor Overloading

We applied constructor overloading in the Customer class. It has three constructors with different parameters, which allow us to create customer objects in different ways.

Public, Private, and Protected

We used private variables to protect data, protected variables for the name and base price in the MenuItem class, and public methods so other classes can access information when needed.

Getters, Setters, and Data Hiding

We used getters and setters to access and update private variables. We also added conditions to prevent invalid values, such as negative prices and negative extra sides.

Method Overloading

We used method overloading in the Customer class through addPoints() and in the Order class through computeTotal(). These methods have the same names but different parameters.

Method Overriding

We applied method overriding in the Beverage, Pastry, and Meal classes. Each class has its own version of computePrice() and getDescription() because every menu item has a different price calculation and description.

Abstraction

We applied abstraction by making MenuItem an abstract class. It contains an abstract method called computePrice(), which is implemented by the Beverage, Pastry, and Meal classes.

Challenges Encountered and How We Solved Them

While doing this activity, we experienced some difficulties, especially when connecting the seven classes and making sure that everything worked properly. There were times when we got confused about which constructor or method to use. We also found it challenging to understand the difference between method overloading and method overriding because they seemed similar to us at first.

Another difficulty was calculating the prices correctly, especially when adding extra charges, student discounts, and vouchers. We also had to make sure that the system would not accept more than three items in one order.

Whenever we encountered errors, we tried to review our code, check the conditions, and run the program again. We also discussed the parts we did not understand and helped each other figure out what needed to be corrected. Some parts took us time to understand, but through patience and repeated testing, we gradually became more familiar with how the program works.

Overall, this activity was a good learning experience for both of us. We realized that programming is not always easy, especially when dealing with several classes and methods, but we learned that mistakes are part of the process. We also learned the importance of understanding the code instead of simply making it run. Most importantly, working together helped us share ideas, solve problems, and learn more about Java and Object-Oriented Programming.

